Reinforcement Learning for Autonomous mobile Robot Navigation: using combined DQN with REINFORCE Optimization

Boukhiar Naima

Faculty of Exact and Applied Sciences, University of Oran1 :Ahmed Ben Bella , Oran Algeria.

Contributing author: naima.boukhiar@ieee.org; m

1 Abstract

This article introduces an IMPROVED hybrid method reinforcement learning algorithm for autonomous mobile robot navigation. Combining the strengths of Deep Q-Network (DQN) and REINFORCE, our approach achieves superior learning efficiency and navigation performance compared to existing RL algorithms. An improved method was presented, leveraging REINFORCE for handling continuous action spaces and exploration, while DQN's neural network enables better generalization in complex state spaces. The experimental results demonstrate that the hybrid method improved the performance, showing significant improvements in success rate using metrics like steps taken and time spent. This advancement in robot navigation holds promise for real-world applications, focusing on the RL algorithm, especially in the navigation aspect.

2 keywords

Mobile robots, Reinforcement Learning, sarsa, Deep Q-Learning, Q-Learning, Autonomous Mobile Robot, Obstacle Avoidance, Robotics, Neural Networks, Hybrid, Navigation.

3 Introduction

A mobile robot is an autonomous machine capable of operating and traversing within a specified environment [25]. Its primary objective is to navigate from a starting position to a designated goal while circumventing any obstacles in its unknown surroundings. Learning is a gradual process of improving system performance based on past experiences. Numerous techniques have been developed to enhance the navigation task of mobile robots, which have widespread applications in various industries such as agriculture, medicine, and clinical treatment. Reinforcement Learning (RL) techniques, one of the most captivating Machine Learning approaches, are highly utilized in robotics for navigation, control, autonomous driving [22], board games [14], resource management, personalized recommendations, and designing robots that learn to accomplish tasks in the real world. The Grid Environment is the most prevalent [19] navigation context.

RL leverages a neural network where a learning agent obtains scalar performance feedback called a reward from its surroundings. The agent learns various RL algorithms [4] and applies them in a different environment after each action to solve the problem of navigation with obstacle avoidance [18]. We express this problem as a maze-search using the Q function [12], which can apply to all robotics-related issues, similar to the maze-search problem.

4 State of Art

Numerous navigation approaches have been proposed and used in robotics this is what helps control the mobile robot performances the previously proposed solution uses the optimal return function, based on the MDP (Markov decision process) [8] [26] we can say that all optimization problems can be classified as RL problems, most of the studies discussed the application of the robot like path planning [16], behavior decision, and motion control remarkable progress has been made in solving robotics problems after the use of the fuzzy [23] [17] logic algorithm, particle swarm optimization **(PSO)**[24], ant colony optimization **(ACO)**[28], artificial bee colony **(ABC)** [11], bacterial foraging optimization **(BFO)**, genetic algorithms **(GA)**[27] those studied focus on the performance of the robot, route planning. these approaches mostly converge to the same optimum reward progress in Deep Learning resulted in a big promotion for recent research on the rapid development of robotics. Although **DL** [13] effectively solves some problems (like robot learning [2], cooperative robot [15]), it still cannot autonomously take actions without the required data. RL then took over where a robot can learn from trial and error by interacting with semi-known or unknown environments. Furthermore, combining RL and neural networks [9] [3] has already shown great significance for future studies. This survey displays more research on reinforcement learning for robotics.

Recent advancements in reinforcement learning for autonomous navigation include the development of a new method combining a multistep update with a double deep Q-network (MS-DDQN) [20]. This approach enhances the navigation ability of mobile robots by using a sophisticated reward system that includes terminal and non-terminal rewards. Experiment results demonstrated higher success rates and improved learning speed compared to other methods like DDQN and prioritized DQN, indicating that MS-DDQN enables mobile robots to navigate autonomously even in unknown complex environments.

Additionally, the design of optimal path planning and load-carrying capacity for manipulators has been investigated using a vibration damping optimization algorithm and a game theory approach [16]. These methods consider the dynamics of both the manipulator and the driving system, using Nash equilibrium strategies to control system disturbances. This ensures minimal changes in load-carrying capacity despite disturbances, showcasing an effective control strategy for fixed-base two-link manipulators.

State-of-the-art of the Existing reinforcement learning algorithms in robotics [1]

RL algorithms determine the specific methods used in training the mobile robot. The data needed for the mobile robot can be generated by the interaction between the robot and the environment. the modern autonomous mobile robots focus on combining RL methods with artificial neural networks like value-based RL, policy-based RL, model-based RL, and deep reinforcement learning (DRL) [29]. In addition, Table 1 shows the method which has been applied to mobile robots.

DQNs (Deep Q Networks), which are deep neural networks, in our technique. It's an RL algorithm using a value-based approach. Each state of the environment would be represented by a collection of pixels, and the agent would be able to do different behaviors depending on whatever condition it was in. Rather than using value iterations as in MDPs to determine the Q-values and find the optimal Q-function, we use a function approximate, i.e. Deep Neural Networks, to estimate the optimal Q-function. The target in Q Learning is determined by the prediction. Q Learning is an off-policy semi-gradient method. To estimate the Q-values for each state-action combination in a particular environment, we shall utilize DQNs. The goal of this network is to approximate the best Q-function possible. The Bellman equation will be satisfied. The network's loss is calculated by comparing the outputted Q-values to the desired Q-values from the Bellman equation's righthand side. The network adjusts weights using Stochastic Gradient Descent and Backpropagation when the loss is determined, and this is how the loss is reduced. During the training of Deep Q Networks, we frequently use the techniques of "Experience Replay" and "Replay Memory." We record all of the agent's experiences et at each time step in a dataset termed replay memory, which is represented as et = (st, at, rt+1, st+1) as a tuple. The dataset for replay memory is taken at random and utilized to train the network; this sampling aids in breaking the association between the two variables. a series of steps

Table 1: Reinforcement Learning Algorithms State-of-the-Art Summary.

RL Algorithm	Category	Training Speed	State Spaces	Action Spaces	Memory Consumed	Policy	Model	Operator
QL	Value-based	Fast	Discrete	Discrete	Large memory not required	Off-policy	Model-free	Q-value
SARSA	Model-free	Average	Discrete	Discrete	Large memory not required	On-policy	Model-free	Q-value
SARSA Trace	Model-free	Fast	Discrete	Discrete	Large memory not required	On-policy	Model-free	Q-value
Q-Trace	Value-based	Average	Discrete	Discrete	Large memory not required	Off-policy	Model-free	Q-value
DQN	Value-based	Slow	Discrete	Discrete	Large memory required	Off-policy	Model-free	Q-value
REINFORCE	Policy-based	Average	Various	Various	Large memory not required	On-policy	Model-free	Policy gradient
(PPO)	Policy Gradient	Fast	Contin	Contin	Large Memory	On-policy	Model-free	Policy Gradient
(SAC)	Actor-Critic	Fast	Contin	Contin	Large Memory	Off-policy	Model-free	Q-value/Q-function
(TD3)	Actor-Critic	Fast	Contin	Contin	Large Memory	Off-policy	Model-free	Q-value/Q-function
Rainbow	Value-based	Average	Discrete	Discrete	Large Memory	Off-policy	Model-free	Q-value
Alpha Z	Value-based	Average	Discrete	Discrete	Large Memory	Self-play	Model-free	Monte Carlo Tree Search

Source: This is an example of table footnote. This is an example of table footnote.

[1]Example for a first table footnote. This is an example of table footnote.

[2]Example for a second table footnote. This is an example of table footnote.

Table 2: Summary Comparison of Reinforcement Learning Algorithms 1

RL Algorithm	Environment	Performance Metric	Achievements
DQN	Atari 2600 Games	Average Score	Achieved state-of-the-art performance on various Atari 2600 games.
QL (Q-Learning)	Various	Average Return	Demonstrated success in solving a wide range of discrete control tasks.
Q-Trace	Various	Average Return	Improved credit assignment and convergence in temporal difference learning.
SARSA	Various	Average Return	Achieved good performance in on-policy control tasks.
SARSA Trace	Various	Average Return	Enhanced learning and credit assignment in on-policy control tasks.
REINFORCE	Various	Average Return	Successfully applied to policy optimization tasks, including continuous control domains.

5 Problem description

A Robot learning to interact with its surroundings is described by the reinforcement learning (RL) Algorithm, model, and environment. The agent gets the state of the environment (the environment presents a scenario to the agent) at each time step, and the agent must pick an appropriate response. The agent receives a reward and a new state a one-time step later (the environment determines if the agent behaved adequately to the state).

All agents aim to maximize the expected cumulative reward, which is the expected total of all rewards earned across all time steps.

6 The Proposed Method

this study focuses on how reinforcement learning improved mobile robot navigation through training different models in different Environments to find the optimal policy Once the agent

Table 3: Summary Comparison of Reinforcement Learning Algorithms 2

Algo	Sample efficiency	Bias	Variance	Scalability	Notes
Q-learning	Low	High	High	Low	Simple, but can be inefficient for large state spaces
SARSA	Low	High	High	Low	Similar to Q-learning, but updates Q-values based on both the current and next state
DQN	High	Low	High	Low	Uses a neural network to approximate Q-values, which makes it more sample efficient than Q-learning
Double DQN	High	Low	Low	Low	Improves upon DQN by estimating Q-values from both the current and next state
Dueling DQN	High	Low	Low	Low	Separates the estimation of state-value and action-value functions, which can improve performance
Rainbow	High	Low	Low	Low	Combines several techniques, including DQN, Double DQN, Dueling DQN, and prioritized experience replay, to improve performance
Policy gradient	High	Low	High	High	Directly optimizes the policy, which can be more sample efficient than Q-learning-based methods
TRPO	High	Low	Low	High	Uses a trust region constraint to ensure that the policy updates are not too large, which can help to prevent overfitting
A2C	High	Low	Low	High	Combines policy gradient with Q-learning to improve performance
PPO	High	Low	Low	High	Similar to TRPO, but uses a different trust region constraint

Table 4: Reinforcement Learning Algorithms Contribution in relevant references

RL Algo	**Article Reference**	**Algorithm Contribution**
Q-Learning	Mnih, V. et al. (2015). Human-level control through deep reinforcement learning.	Applied deep neural networks to Q-learning, achieving state-of-the-art performance on several Atari 2600 games.
Q- Traces	Harutyunyan, A., Munos, R. Bellemare, M. G. (2016). Q(): Generalized Off-Policy Policy Evaluation.	Introduced Q(), a generalized version of Q-learning with eligibility traces that allow for flexible credit assignment.
SARSA	Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction.	Provides a comprehensive introduction to reinforcement learning, including the SARSA algorithm and its variants.
SARSA Traces	Seijen, H. V., Sutton, R. S., Mahmood, A. R., Pilarski, P. M., & Machado, M. C. (2017). True Online Temporal-Difference Learning.	Introduced True Online SARSA(), an efficient algorithm for SARSA with eligibility traces that allow for online updates and efficient function approximation.
DQN	Wang, Z. et al. (2015). Dueling network architectures for deep reinforcement learning.	Introduced the concept of dueling networks in DQN, improving the estimation of state and action values by decoupling them.
REINFORCE	Schulman, J. et al. (2015). Trust Region Policy Optimization.	Presented the Trust Region Policy Optimization (TRPO) algorithm, an extension of REINFORCE that optimizes policies through trust region updates, leading to improved sample efficiency and stability.

determines the optimal action-value function Q, it can quickly obtain an optimal policy that has the potential to be applied on real-world robot

7 Reinforcement learning

is a Learning process to improve the performance of a system based on its past experiences(trial and error) [21]interacting with its environment. where from a state or situation **S** in the environment, the agent selects and performs an action that causes a transition to state **S'**. He receives in return a reinforcement signal r, which can be a reward or punishment. The purpose of this learning experience is to maximize future rewards. [7]

8 contribution

8.1 Implications:

The simulation results illustrate the effectiveness of the proposed hybrid reinforcement learning algorithm in various complex environments. Each scenario presents unique challenges, such as maze-like structures, obstacles of different shapes and densities, and massive spaces with sparse obstacles. The successful navigation in these digressive scenarios highlights the algorithm's performance.

8.2

Strengths:

- Versatility:The algorithm performs effectively on a whole range of environments, indicating its flexibility in handling different navigation challenges
- Efficiency:The neural network architecture appears to efficiently process sensor data and make decisions in real-time, as witnessed by the successful navigation paths.
- Durability: The ability to handle scenarios with high obstacle density (e.g., scenarios 6 and 9) showcases the algorithm's capability against potential collisions

8.3 Challenges:

Computational Complexity: The complexity of the neural network and the reinforcement learning algorithm may result in high computational requirements, potentially limiting real-time application in resource-constrained environments. **Spontaneities:**While the algorithm performs well in simulated environments, its performance in real-world scenarios with unpredictable elements may vary. **Preparation drills:**The training process for reinforcement learning can be overwhelming, especially for complex domains, which might delay deployment in practical applications.

8.4 Real-World Applications:

Autonomous Vehicles: The hybrid algorithm can be applied to self-driving cars for navigating through urban environments with dynamic obstacles. **Warehouse Robots:**In automated warehouses, robots can use this algorithm to navigate efficiently among shelves and other obstacles. **Search and Rescue:**Robots equipped with this navigation algorithm can be deployed in disaster-stricken areas to search for survivors while avoiding debris and unstable structures.

Reinforcement learning Algorithms used in this paper

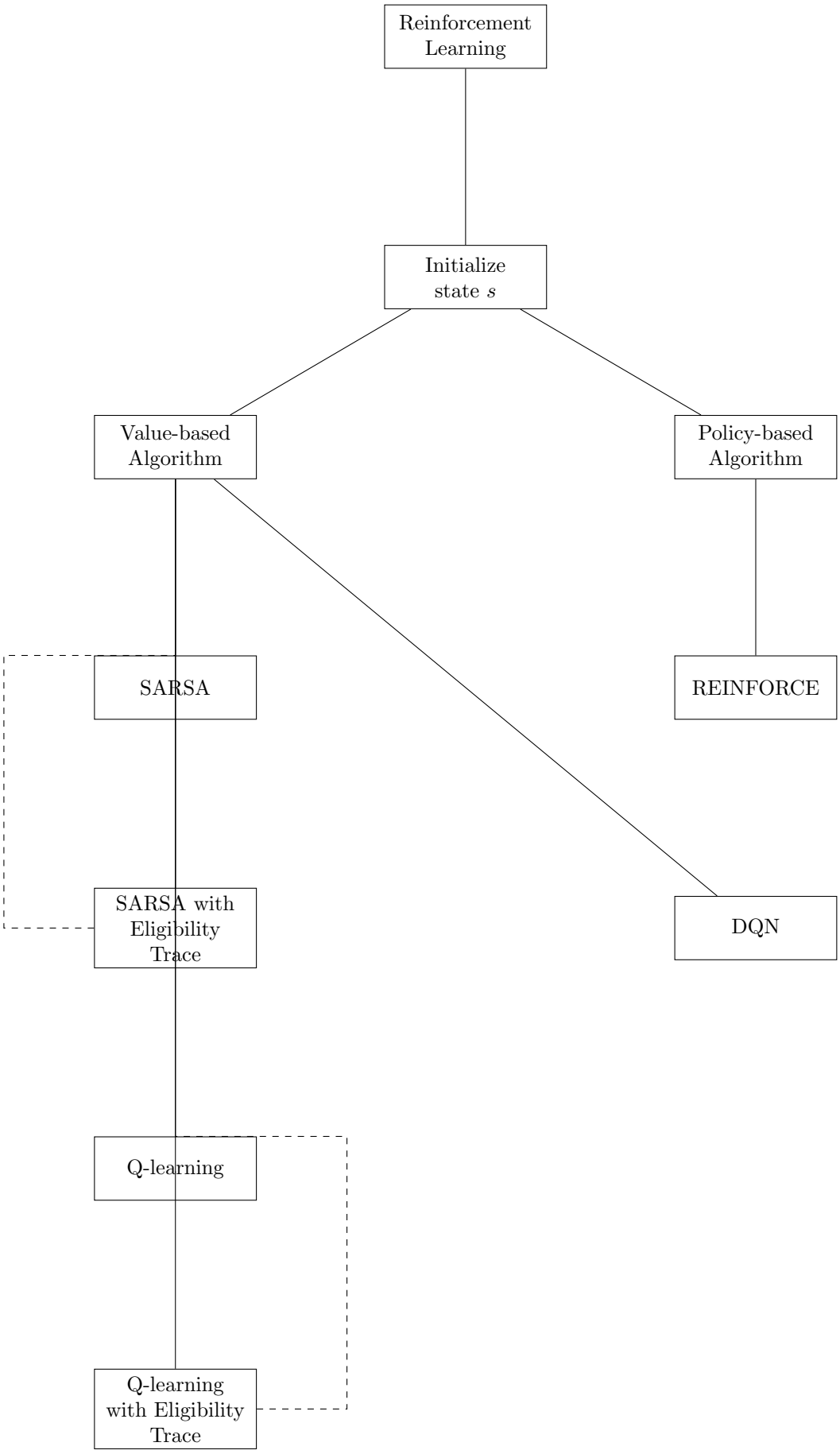


Fig. 1: Diagram representation of Reinforcement Learning Methods implemented in this paper

As shown in the previous figure represents the integration of value-based and policy-based algorithms in reinforcement learning. It showcases the sequential steps involved, starting with state initialization. The value-based algorithm path includes SARSA, SARSA with Eligibility Trace, Q-learning, and Q-learning with Eligibility Trace, focusing on value estimation. The policy-based algorithm path includes REINFORCE, directly learning a parameterized policy. The integration allows leveraging the strengths of both approaches, improving learning and decision-making in complex environments. The inclusion of DQN highlights the use of deep neural networks for value approximation. The flowchart provides a visual representation of the unified framework, demonstrating the coexistence and integration of value-based and policy-based algorithms for enhanced reinforcement learning performance

9 MDP: Markov Decision Process

Markov Decision Process is a framework used to represent a reinforcement learning problem Markov Decision Process is Markov Reward Process with a decision. Reinforcement learning uses MDPs where the probabilities or rewards are unknown. For this purpose it is useful to define a further function, which corresponds to taking the action a and then continuing optimally (or according to whatever policy one currently has: $\mathrm{Q(s,a)} = \sum_{s'} P_a(s,s')(R_a(s,s') + \gamma V(s'))$.

10 TD Method: Temporal Difference

With temporal Difference (TD) methods instead of updating the Q-table at the end of each episode, it will be updated after every time step which works better than the classical approaches when it comes to the predicted value.

10.1 Mathematics Equations: the Q-Function

Q-function uses the Bellman equation and takes two inputs: state (s) and action (a)

$$Q^{new}(s_t,a_t) \leftarrow \underbrace{Q(s_t,a_t)}_{\text{old value}} + \underbrace{\alpha}_{\text{learning rate}} \cdot \overbrace{\Bigg(\underbrace{\underbrace{r_t}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{\max_a Q(s_{t+1},a)}_{\text{estimate of optimal future value}}}_{\text{new value (temporal difference target)}} - \underbrace{Q(s_t,a_t)}_{\text{old value}}\Bigg)}^{\text{temporal difference}} \tag{1}$$

The Bellman equation is used by Q-learning algorithms to learn the Q-function. Q-learning algorithms start with a random Q-function and then iterative update the Q-function using the Bellman equation. As the Q-function is updated, it gets closer and closer to the optimal Q-function. Q-learning is a temporal-difference (TD) learning algorithm. TD learning algorithms update the Q-function based on the immediate reward and the estimated value of the next state. SARSA is a simultaneous TD learning algorithm, while DQN is a deep Q-learning algorithm.

we implemented several algorithms that use the Q-function, including Q-learning, SARSA, and DQN. These algorithms all use the Bellman equation to learn the Q-function, but they differ in how they update the Q-function.

11 RL applied Algorithms

11.1 Q-Learning Pseudocode [10]

Q-learning is a temporal-difference (TD) learning algorithm that updates the Q-function based on the immediate reward and the estimated value of the next state. Q-learning is a simple and efficient algorithm, but it can be slow to learn in complex environments.

Algorithm 1 Q-learning: Learn function $Q : \mathcal{X} \times \mathcal{A} \to \mathbb{R}$

States $\mathcal{X} = \{1, \ldots, n_x\}$
Actions $\mathcal{A} = \{1, \ldots, n_a\}$, $A : \mathcal{X} \Rightarrow \mathcal{A}$
Reward function $R : \mathcal{X} \times \mathcal{A} \to \mathbb{R}$
Black-box (probabilistic) transition function $T : \mathcal{X} \times \mathcal{A} \to \mathcal{X}$
Learning rate $\alpha \in [0, 1]$, typically $\alpha = 0.1$
Discounting factor $\gamma \in [0, 1]$
procedure QLEARNING($\mathcal{X}$, A, R, T, α, γ)
Initialize $Q : \mathcal{X} \times \mathcal{A} \to \mathbb{R}$ arbitrarily **while** Q *is not converged* **do**
end
Start in state $s \in \mathcal{X}$ **while** s *is not terminal* **do**
end
Calculate π according to Q and exploration strategy (e.g. $\pi(x) \leftarrow_a Q(x, a)$)
$a \leftarrow \pi(s)$
$r \leftarrow R(s, a)$ ▷ Receive the reward
$s' \leftarrow T(s, a)$ ▷ Receive the new state
$Q(s', a) \leftarrow (1 - \alpha) \cdot Q(s, a) + \alpha \cdot (r + \gamma \cdot \max_{a'} Q(s', a'))$
$s \leftarrow s'$

return Q
end procedure

11.2 Q_Eligibilty Trace pseudo-code

Q-Traces is a modification of Q-learning that uses eligibility traces to improve the learning speed. Eligibility traces are a way of keeping track of which state-action pairs have been recently visited so that they can be updated more heavily when the Q-function is updated. Q-Traces can significantly improve the learning speed of Q-learning, especially in complex environments.

Algorithm 2 Q-Eligibility Trace

Require:
Initialize $Q(s, a)$ arbitrarily and $e(s, a) = 0$, for all s, a
Repeat (for each episode):
1: Initialize s, a **repeat**for each step of the episode
2:
until :;
Take action a, observe r, s'
3: Choose a' from s' using policy derived from Q (e.g., ϵ-greedy)
4: $a^* \leftarrow \arg\max_b Q(s', b)$ (if a ties for the max, then $a^* \leftarrow a'$)
5: $\delta \leftarrow r + \gamma Q(s', a') - Q(s, a^*)$
6: $e(s, a) \leftarrow e(s, a) + 1$ **for *all*** s, a **do**
7:
end
$Q(s, a) \leftarrow Q(s, a) + \alpha\delta e(s, a)$ **if** $a' = a^*$ **then**
8:
end
$e(s, a) \leftarrow \gamma\lambda e(s, a)$ **else**
9:
end
$e(s, a) \leftarrow 0$
10:
11:
12: $s \leftarrow s'$; $a \leftarrow a'$
13: s is terminal

11.3 Sarsa Pseudo-code

SARSA is a simultaneous TD learning algorithm that updates the Q-function based on the immediate reward and the action that was actually taken in the next state. SARSA is more efficient than Q-learning, but it can be more prone to overfitting.

Algorithm 3 SARSA: Learn function $Q : \mathcal{X} \times \mathcal{A} \to \mathbb{R}$

Require:
Sates $\mathcal{X} = \{1, \ldots, n_x\}$
Actions $\mathcal{A} = \{1, \ldots, n_a\}$, $A : \mathcal{X} \Rightarrow \mathcal{A}$
Reward function $R : \mathcal{X} \times \mathcal{A} \to \mathbb{R}$
Black-box (probabilistic) transition function $T : \mathcal{X} \times \mathcal{A} \to \mathcal{X}$
Learning rate $\alpha \in [0, 1]$, typically $\alpha = 0.1$
Discounting factor $\gamma \in [0, 1]$
$\lambda \in [0, 1]$: Trade-off between TD and MC
procedure SARSA($\mathcal{X}$, A, R, T, α, γ, λ)
Initialize $Q : \mathcal{X} \times \mathcal{A} \to \mathbb{R}$ arbitrarily **while** Q *is not converged* **do**
end
Select $(s, a) \in \mathcal{X} \times \mathcal{A}$ arbitrarily **while** s *is not terminal* **do**
end
$r \leftarrow R(s, a)$ ▷ Receive the reward
$s' \leftarrow T(s, a)$ ▷ Receive the new state
Calculate π based on Q (e.g. epsilon-greedy)
$a' \leftarrow \pi(s')$
$Q(s, a) \leftarrow (1 - \alpha) \cdot Q(s, a) + \alpha \cdot (r + \gamma Q(s', a'))$
$s \leftarrow s'$
$a \leftarrow a'$

return Q
end procedure

11.4 Sarsa Eligibity Pseudo-code [5]

SARSA Traces is a modification of SARSA that uses eligibility traces to improve the learning speed. SARSA Traces can significantly improve the learning speed of SARSA, especially in complex environments.

Algorithm 4 SARSA(λ): Learn function $Q : \mathcal{X} \times \mathcal{A} \to \mathbb{R}$

Require:
Sates $\mathcal{X} = \{1, \ldots, n_x\}$
Actions $\mathcal{A} = \{1, \ldots, n_a\}$, $A : \mathcal{X} \Rightarrow \mathcal{A}$
Reward function $R : \mathcal{X} \times \mathcal{A} \to \mathbb{R}$
Black-box (probabilistic) transition function $T : \mathcal{X} \times \mathcal{A} \to \mathcal{X}$
Learning rate $\alpha \in [0, 1]$, typically $\alpha = 0.1$
Discounting factor $\gamma \in [0, 1]$
$\lambda \in [0, 1]$: Trade-off between TD and MC
procedure QLEARNING($\mathcal{X}$, A, R, T, α, γ, λ)
Initialize $Q : \mathcal{X} \times \mathcal{A} \to \mathbb{R}$ arbitrarily
Initialize $e : \mathcal{X} \times \mathcal{A} \to \mathbb{R}$ with 0. ▷ eligibility trace **while** Q *is not converged* **do**
end
Select $(s, a) \in \mathcal{X} \times \mathcal{A}$ arbitrarily **while** s *is not terminal* **do**
end
$r \leftarrow R(s, a)$
$s' \leftarrow T(s, a)$ ▷ Receive the new state
Calculate π based on Q (e.g. epsilon-greedy)
$a' \leftarrow \pi(s')$
$e(s, a) \leftarrow e(s, a) + 1$
$\delta \leftarrow r + \gamma \cdot Q(s', a') - Q(s, a)$ **for** $(\tilde{s}, \tilde{a}) \in \mathcal{X} \times \mathcal{A}$ **do**
end
$Q(\tilde{s}, \tilde{a}) \leftarrow Q(\tilde{s}, \tilde{a}) + \alpha \cdot \delta \cdot e(\tilde{s}, \tilde{a})$
$e(\tilde{s}, \tilde{a}) \leftarrow \gamma \cdot \lambda \cdot e(\tilde{s}, \tilde{a})$

$s \leftarrow s'$
$a \leftarrow a'$

return Q
end procedure

11.5 Deep Q-Learning pseudo-code [6]

DQN is a deep Q-learning algorithm that uses a deep neural network to represent the Q-function. Deep neural networks are able to learn complex relationships between states and actions, which makes them well-suited for reinforcement learning tasks. DQN has been shown to be very effective in a variety of reinforcement learning tasks, including playing Atari games.

Algorithm 5 Deep Q-Network (DQN)

Initialize replay memory $\mathcal{D}$ with capacity N
Initialize action-value function Q with random weights θ
Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$ **for** *episode* $= 1$ *to* M **do**
    end
    Initialize state s **for** $t = 1$ *to* T **do**
    end
    With probability ϵ select a random action a_t
Otherwise select $a_t = \arg\max_a Q(s, a; \theta)$
Execute action a_t, observe reward r_t and new state s'
Store transition (s, a_t, r_t, s') in $\mathcal{D}$
Sample random mini-batch of transitions (s_i, a_i, r_i, s'_i) from $\mathcal{D}$
Set $y_i = \begin{cases} r_i & \text{for terminal } s'_i \\ r_i + \gamma \max_{a'} \hat{Q}(s'_i, a'; \theta^-) & \text{for non-terminal } s'_i \end{cases}$
Perform a gradient descent step on $(y_i - Q(s_i, a_i; \theta))^2$ with respect to θ
Every C steps, update target network: $\theta^- = \theta$
Set $s = s'$

11.6 REINFORCE Pseudo-code

REINFORCE is a policy gradient algorithm that updates the policy directly, without explicitly estimating the Q-function. REINFORCE works by using the policy to generate a trajectory of states and actions, and then backpropagating the rewards through the trajectory to update the policy. REINFORCE is a powerful algorithm, but it can be slow to learn in complex environments.

Algorithm 6 REINFORCE

Initialize policy parameters θ **for** *episode* $= 1$ *to* M **do**
    end
    Collect trajectory using policy $\pi(\cdot; \theta)$
Initialize empty lists: G (returns), R (rewards), A (actions)
Initialize the initial state s_0 **repeat**
    S
    until
;
    ample action a_t from the policy $\pi(\cdot; \theta)$
Take action a_t, observe reward r_t and new state s_{t+1}
Append r_t to R, a_t to A, and s_{t+1} to S
$t \leftarrow t + 1$
terminal state reached
Compute the returns G from the rewards R **for** $t = 0$ *to* $T - 1$ **do**
    end
    Compute the discounted future return G_t
Compute the policy gradient $\nabla_\theta \log \pi(a_t|s_t; \theta)$
Update policy parameters: $\theta \leftarrow \theta + \alpha \cdot G_t \cdot \nabla_\theta \log \pi(a_t|s_t; \theta)$
terminal state reached

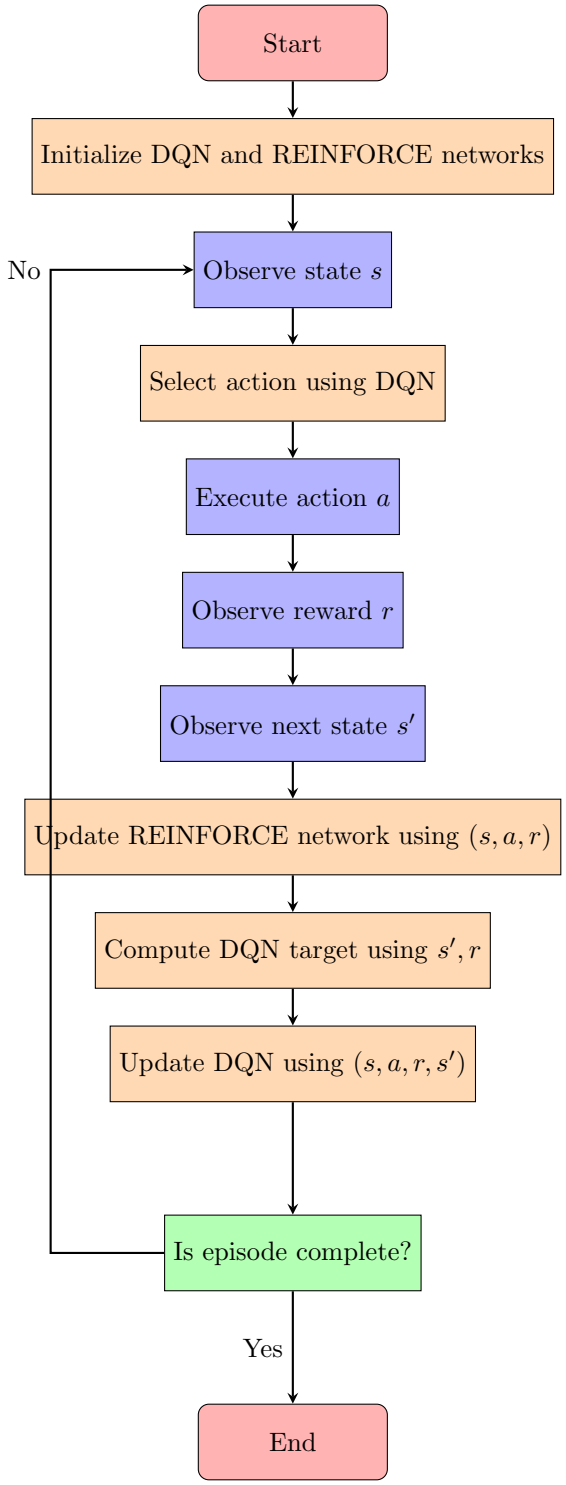


Fig. 2: The flowchart of the Combined DQN-REINFORCE algorithm

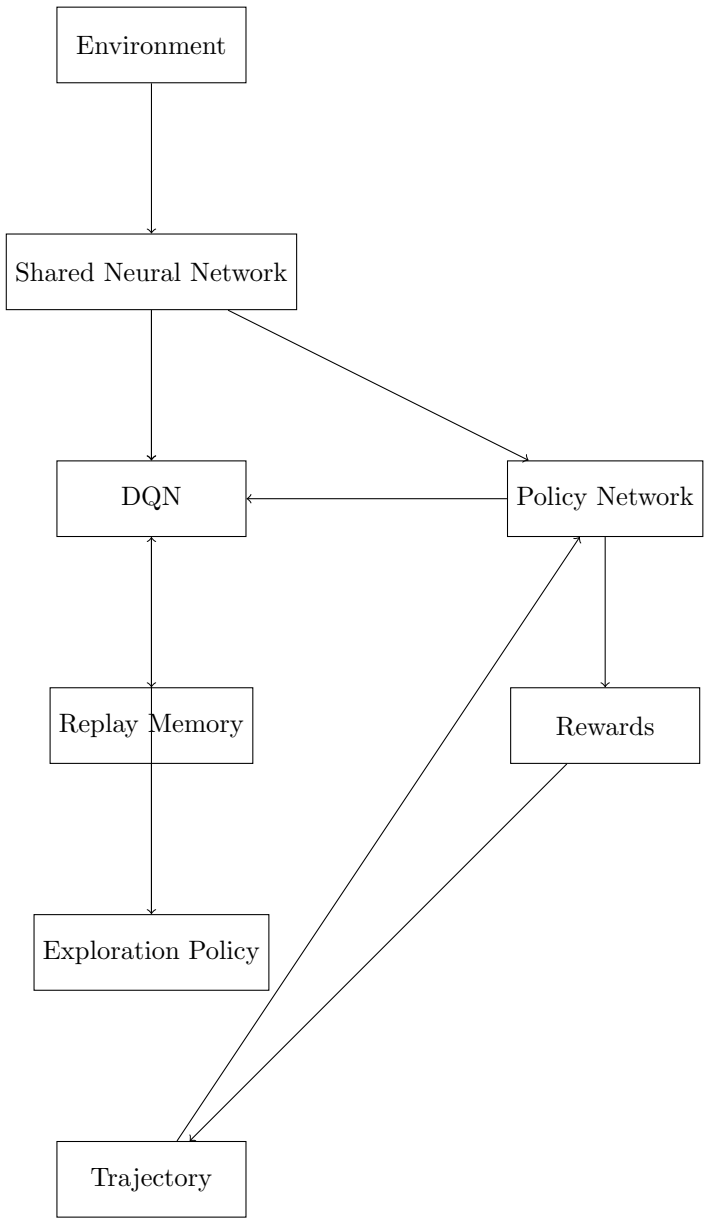


Fig. 3: Combined DQN and REINFORCE Architecture with Shared Neural Networks

The provided flowchart shows the combination of DQN (Deep Q-Network) and REINFORCE (Monte Carlo Policy Gradient) algorithms in a joint reinforcement learning framework. The flowchart shows the interaction between agent, environment, reward, condition." , action, and a shared neural network." Agent gets current state from the environment, joint DQN." -selects an action based on the REINFORCE process, and checks the corresponding reward. State, action, and reward are then sent to a shared root, which... processes information and generates updated values for state-action pairs DQN part of the algorithm Value-based -Focuses on learning, uses the Bellman equation and a small modification to create a neural network new parameters.On the other hand, the REINFORCE phase emphasizes policy-based learning, using the REINFORCE algorithm with the same small set of transitions to update policy parameters. The combination of these two policies enables you to take the approach that encompasses cost estimation and optimization of programs and advanced learning and decision-making in reinforcement learning

12 Simulation results

The simulated results are plotted to compare the results of the RL algorithms paying particular attention to the average training time, average number of episodes, and reward evolution. The The lesser the number of iterations, the more optimal the algorithm's performance

Parameter	Value
maximum number of iterations	10000
maximal number of episodes	3000
learning rate(α)	0.01
discount factor(γ)	0.90
exploration rate	$0,10$
exploration decay	$0,995$
Batch size	64

Table 5: hyper-parameters used for this simulation.

13 Setting

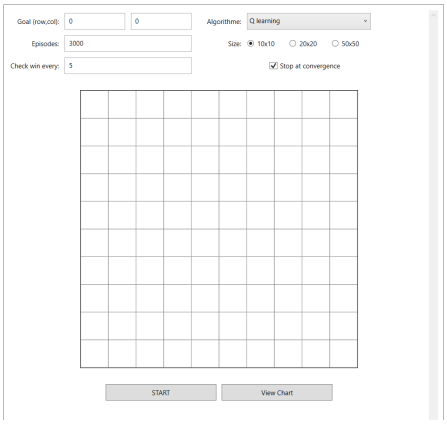


Fig. 4: The Framework settings .

To put our work to the test, we applied the same algorithms to different environments to obtain and evaluate different models that we get as results. We have designed several trials to simulate the robot navigation through the Robot World.

- State Grid of 10X10, 20x20,50x50 cell
- Action The robot can move in 4 directions (up, down, left, right) to move around
- Reward 1.0 for the robot to reach the final state the state that has the maximum reward is the goal state
- Penalty -1 every time the robot finds an obstacle
- Environment is a grid where the robot can do a sequence of episodes from the starting position to the goal position

14 Technical Details

The simulations were implemented using a computer meeting the Minimum Hardware Requirements for RL framework:config I7 12 th generation 12 core 32 Ram NVME 512,RTX 4060 .

we present a new neural network framework for C ♯ While both Python and C♯ are popular choices for RL development, we opted for C♯ due to its ability to provide greater control over code modifications, which is crucial for optimizing algorithms and performance. Additionally, C♯'s tight integration with Unity facilitates a smoother transition to 3D environments, ultimately paving the way for real-world robot navigation applications.

C♯ Simulation for Mobile Robot navigation

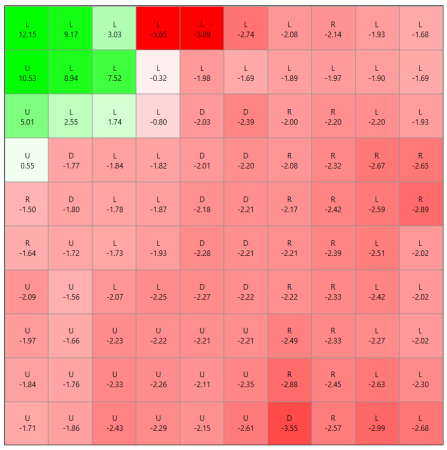


Fig. 5: A capture of The Qtable.

The Q-table

The Q-table has columns labeled "L", "U", "D", and "R".to represent different directions (Left, Up, Down, Right), the data displayed is the Q-value changing from negative to positive based on the reward the robot is receiving

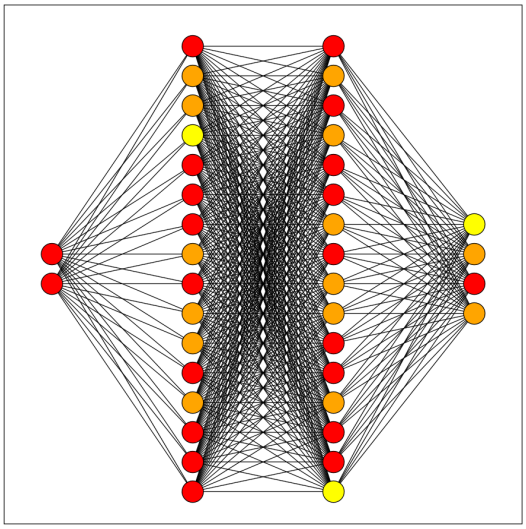


Fig. 6: A capture of the Neural Network architect

The Neural Network

Our agent leveraged a two-layer neural network (16 neurons each) to estimate Q-values for action selection. The network received the current location (2D vector) as input and predicted Q-values for all possible actions (output layer: 4 neurons). ReLU activation in hidden layers facilitated learning, while the linear output ensured a direct representation of Q-values. We trained using ADAM optimization, iteratively computing Q-values, comparing them with target values (considering future rewards), and updating network weights until convergence. The agent then chose the action with the highest predicted Q-value, maximizing long-term reward.

15 Evaluation and experimental results

we now describe the agent's behavior and evolution in different scenarios

15.1 The value-based Algorithms simulation scenarios

this image illustrates the agent performance in an environment with specific settings as shown in the fig using the Value-based algorithm(Q-Learning, Q-learning Trace, Sarsa,sarsa trace) the estimated value increases as the agents get closer to the goal while the trajectory goes from lighter to darker Green

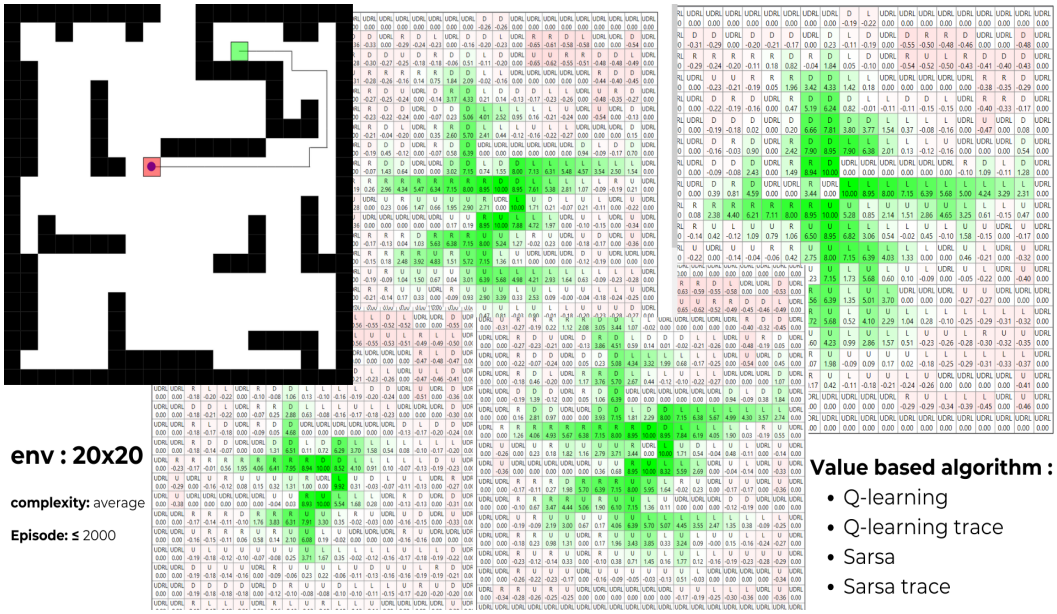


Fig. 7: Value Based Algorithm simulation

15.2 Policy based Algorithms simulation scenarios

this image illustrates the agent performance in similar settings using a policy-based algorithm (REINFORCE) the estimated policy increases as the agents get closer to the goal while the trajectory goes from negative to positive reward as for REINFORCE, the state is based on random values that maximize the reward where each action probability is directly updated to get an optimal estimated policy

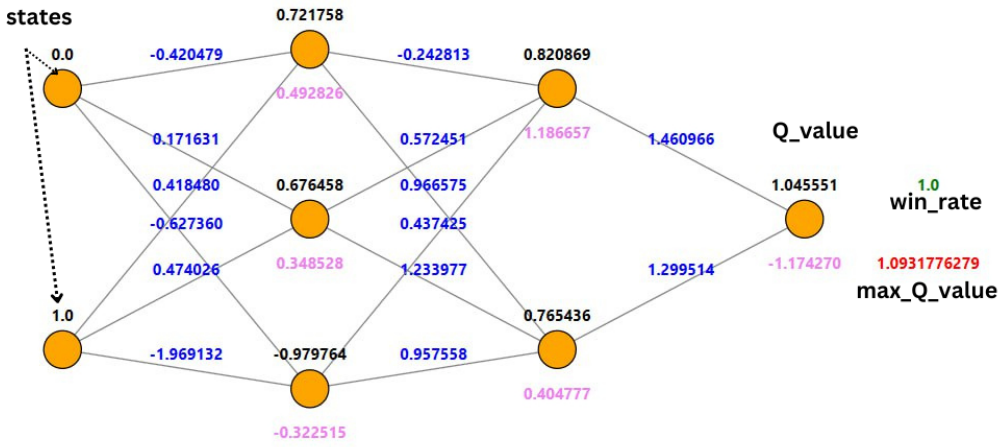


Fig. 8: Policy-based Algorithm simulation

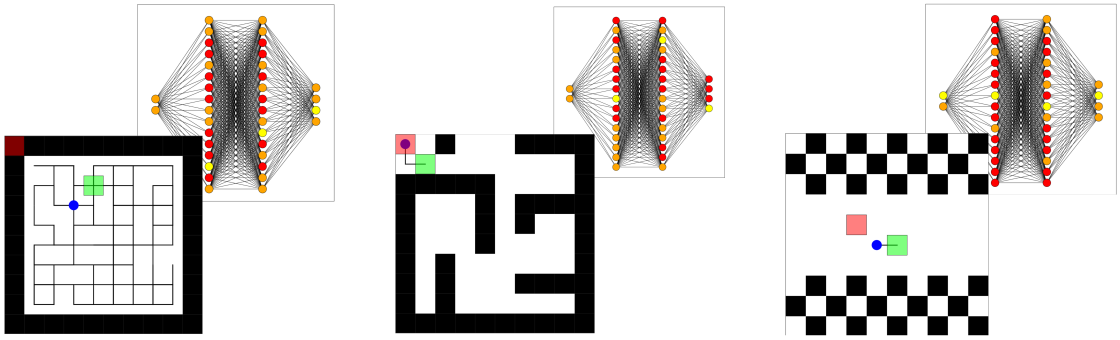
15.3 Hybrid (combined REINFORCE with DEEP QL)

The detailed parameters used in Hybrid method Simulation are listed in the Table below

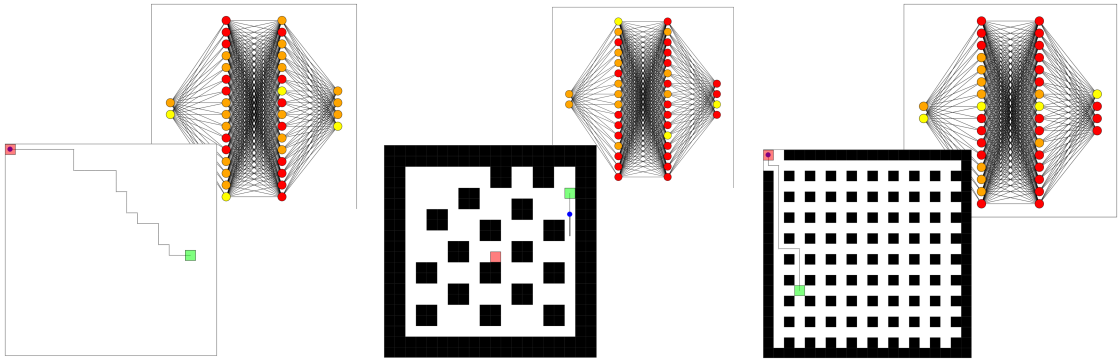
Scenario 1 **Grid:** 10x10 **Obstacles:** Empty **Episode:** ≤ 1000 **Time Spent:** **Start/Goal Position:** Random (Not Fixed)/ (0,0)	**Scenario 2** **Grid:** 10x10 **Obstacles:** Average **Episode:** ≤ 1000 **Time Spent:** **Start-Goal Position:** Random (Not Fixed)/ (0,0)	**Scenario 3** **Grid:** 10x10 **Obstacles:** Complex **Episode:** ≤ 1000 **Time Spent:** **Start-Goal Position:** Random (Not Fixed)/ (0,0)
Scenario 4 **Grid:** 20x20 **Obstacles:** Empty **Episode:** ≤ 2000 **Time Spent:** **Start-Goal Position:** Random (Not Fixed)/ (9,9)	**Scenario 5** **Grid:** 20x20 **Obstacles:** Average **Episode:** ≤ 2000 **Time Spent:** **Start-Goal Position:** Random (Not Fixed)/ (9,9)	**Scenario 6** **Grid:** 20x20 **Obstacles:** Complex **Episode:** ≤ 2000 **Time Spent:** **Start-Goal Position:** Random (Not Fixed)/ (9,9)
Scenario 7 **Grid:** 50x50 **Obstacles:** Empty **Episode:** ≤ 3000 **Time Spent:** **Start/Goal Position:** Random (Not Fixed)/ (49,49)	**Scenario 8** **Grid:** 50x50 **Obstacles:** Average **Episode:** ≤ 3000 **Time Spent:** **Start/Goal Position:** Random (Not Fixed)/ (49,49)	**Scenario 9** **Grid:** 50x50 **Obstacles:** Complex **Episode:** ≤ 3000 **Time Spent:** **Start/Goal Position:** Random (Not Fixed)/ (49,49)

Fig. 9: the Hybrid method Settings.

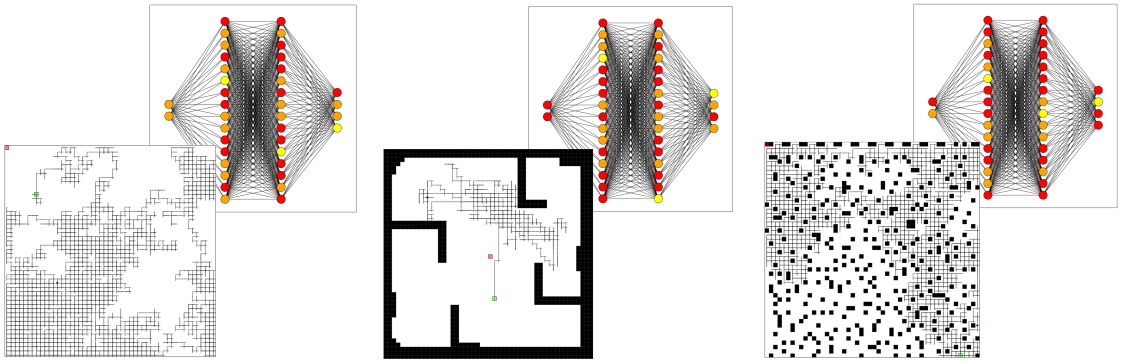
As we shift our focus to the hybrid model developed in our article we showcase different scenarios to demonstrate the improvement of the different simulations that have been done On the following, we present the improvements on the hybrid algorithm with an input layer of the neural network of two neurons, one for each coordinate. The hidden dense layers of the neural network of 16 neurons each and an output layer of the neural network of four neurons, one for each Q-value. The neural network used the ADAM optimizer and the rectified linear unit (ReLU) activation function for the hidden layers, and the linear activation function for the output layer.



(a) scenario 1 (b) scenario 2 (c) scenario 3



(d) scenario 4 (e) scenario 5 (f) scenario 6



(g) scenario 7 (h) scenario 8 (i) scenario 9

Fig. 10: A visual representation of the robot navigation in different Framework using the hybrid method

Scenario 1

- Description: A simple grid environment featuring a straightforward path from the start (red) to the goal (green).
- Significance: This scenario evaluates the fundamental navigation capabilities and learning efficiency of the autonomous mobile robot in an uncomplicated environment.
- Initial Conditions: The robot initiates from the red square and is required to navigate towards the green square.
- Objective: To reach the goal utilizing the shortest possible path with minimal collisions. Challenges: Minimal, as the environment is designed to assess basic navigational functionality.

Scenario 2

- Description: A maze-like environment characterized by narrow pathways and multiple dead-ends.

- Significance: This scenario assesses the robot's ability to navigate through complex and constrained spaces. Initial Conditions: The robot starts at the red square and must traverse the maze to reach the green square.
- Objective: To successfully navigate through the maze and reach the goal.
- Challenges: Avoiding dead-ends and selecting the optimal path in a convoluted maze.

Scenario 3

- Description: A checkered environment consisting of alternating black and white squares.
- Significance: This scenario examines the robot's capability to recognize and navigate through a patterned environment. Initial Conditions: The robot begins at the red square with the aim of reaching the green square.
- Objective: To navigate through the checkered pattern and reach the goal.
- Challenges: Identifying the correct path amidst a visually confusing pattern.

Scenario 4

- Description: An environment featuring descending step-like obstacles.
- Significance: This scenario tests the robot's ability to manage environments with elevation changes or drop-offs.
- Initial Conditions: The robot initiates at the red square and needs to navigate down the steps to the green square.
- Objective: To successfully navigate down the steps and reach the goal. Challenges: Managing elevation changes and avoiding falling off the steps.

Scenario 5

- Description: A grid populated with multiple randomly placed obstacles.
- Significance: This scenario evaluates the robot's obstacle avoidance capabilities within a cluttered environment.
- Initial Conditions: The robot starts at the red square and must navigate around the obstacles to reach the green square.
- Objective: To avoid collisions and efficiently reach the goal.
- Challenges: Detecting and maneuvering around multiple obstacles.

Scenario 6

- Description: A densely packed environment with narrow pathways between obstacles.
- Significance: This scenario tests the robot's precision in navigating through confined spaces.
- Initial Conditions: The robot begins at the red square and aims to reach the green square through narrow pathways.
- Objective: To navigate through narrow spaces without collisions.
- Challenges: High precision required to avoid obstacles within tight spaces.

Scenario 7

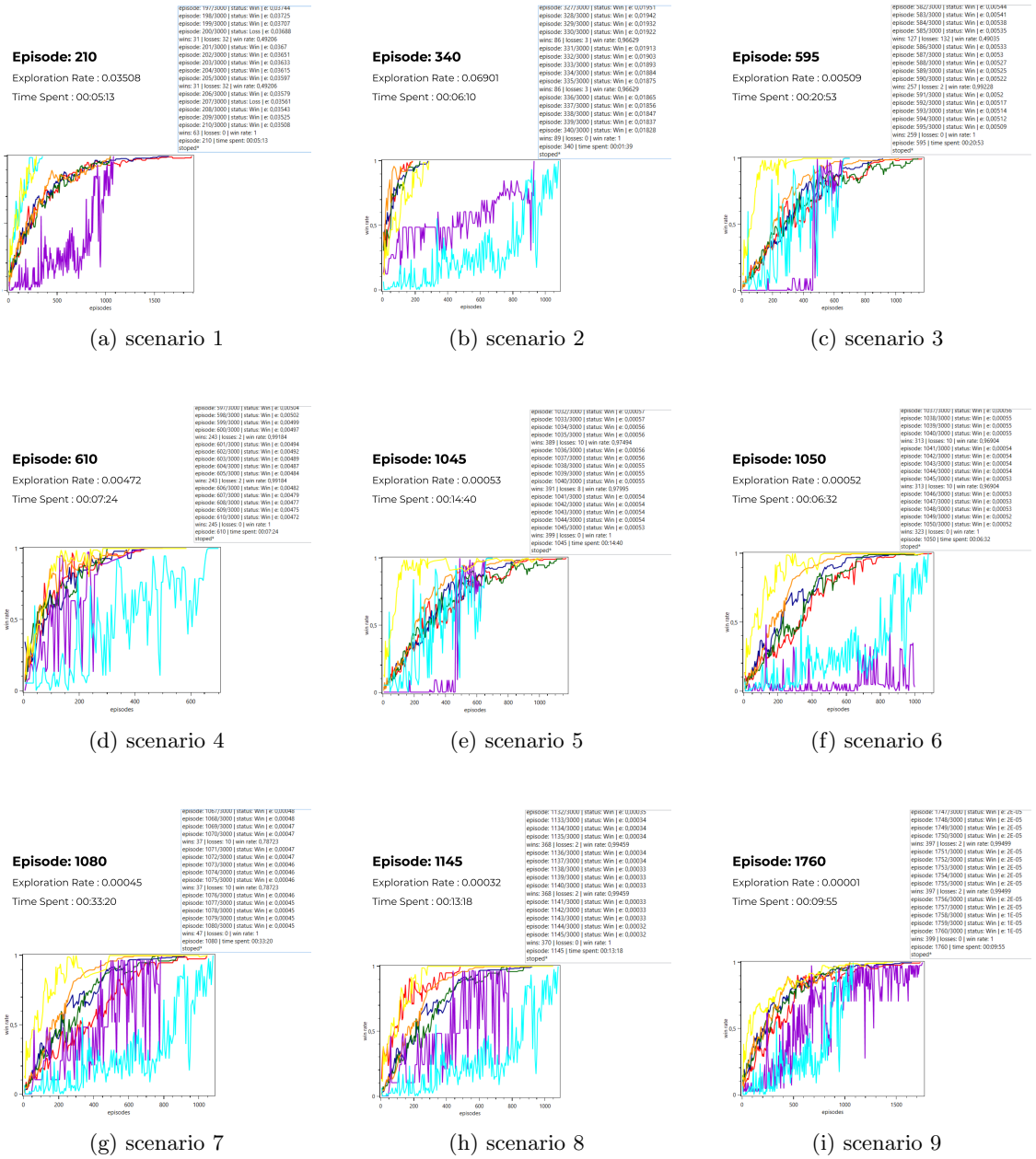
- Description: A highly complex environment with randomly distributed obstacles creating a maze-like structure.
- Significance: This scenario assesses the robot's advanced navigation and decision-making skills in a highly intricate environment.
- Initial Conditions: The robot initiates from the red square and must navigate through a complex maze to reach the green square.
- Objective: To successfully navigate the maze and reach the goal.
- Challenges: High complexity and decision-making required to identify the correct path.

Scenario 8

- Description: A maze comprising various room-like sections and narrow corridors.
- Significance: This scenario evaluates the robot's capability to navigate through structured environments consisting of rooms and corridors.
- Initial Conditions: The robot starts at the red square and needs to navigate through rooms and corridors to reach the green square.
- Objective: To navigate through the maze efficiently and reach the goal. Challenges: Avoiding dead-ends and efficiently maneuvering through the maze structure.

Scenario 9

- Description: An environment with a high density of randomly placed small obstacles.
- Significance: This scenario tests the robot's fine navigation and collision avoidance capabilities within a highly cluttered environment.
- Initial Conditions: The robot initiates from the red square and aims to reach the green square through a dense obstacle field.
- Objective: To navigate through the dense field of obstacles without collisions.



(a) scenario 1 (b) scenario 2 (c) scenario 3

(d) scenario 4 (e) scenario 5 (f) scenario 6

(g) scenario 7 (h) scenario 8 (i) scenario 9

Fig. 11: A visual representation of the robot navigation in different environments using combined REINFORCE -DQN Method

- Challenges: High precision and advanced obstacle avoidance required.

16 A comparative study of previous algorithms

In this section, we analyze the main experimental results. First, we show comprehensive comparative study to evaluate the performance of our hybrid method vs value-based algorithms (e.g, Q-learning, QTrace, Sarsa, Sarsa trace) and policy-based algorithms (e.g., REINFORCE). Our analysis focuses on the main metrics, including cumulative reward, win rate, number of steps taken, number of episodes required for convergence, and the time spent during the learning process. These charts present a synoptic view of the performance of each method for the Evaluated parameters. the hybrid method explores a relatively higher Q-estimated value compared to the other methods expanding further the possibilities of improvements for each action (charts on the Superior right). The chart shows the win rate of different algorithms, with the hybrid method (yellow) having the highest win rate in most experiments **X-axis:** labeled "episodes" referring to training episodes for the algorithms. It spans from 0 to 3000 episodes. **Y-axis:** labeled "win rate"

or (successful rate)with values ranging from 0 to 1. each color refers to a different algorithm so overall we can conclude that The yellow line (the Hybrid method) is consistently close to 1 y-ax, throughout the 3000 episodes. This says that the hybrid y method achieved a win rate close to 100 across most episodes. In contrast, the win rate for the other methods appears to be lower throughout the different experiments.

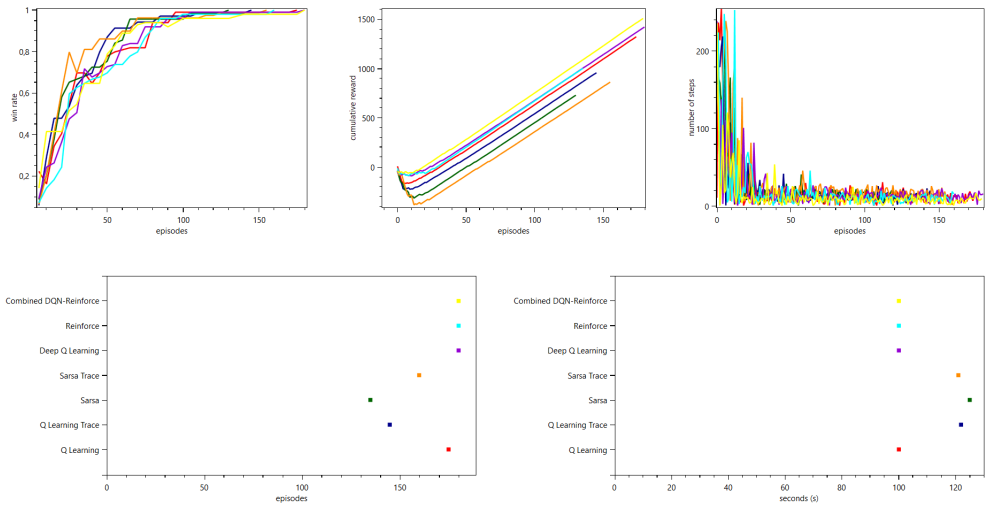


Fig. 12: A Comparative Analysis of Hybrid Method, Value-Based, and Policy-Based using different metrics

Performance metrics: As we can see the performance in the previous picture which we can say it shows the performance of the robot through different algorithms in a perfect environment so far as we can see in the values shown (win rate, reward , number of steps, time spent, number of episodes, exploration rate The graphs show that the "Combined DQN Reinforce" performs better than the other algorithms in most metrics Specifically: the reward graph shows that the robot has higher feedback compared to the other algorithms and a lower number of steps compared to the other algorithms, suggesting the agent reached the goal in fewer steps using the hybrid

17 Discussion

To ensure a controlled comparison of reinforcement learning (RL) algorithms for mobile robot navigation, we adopted a standardized training approach. All algorithms were trained with identical hyperparameters, meticulously chosen through N iterations of tuning for each algorithm. This approach isolated the intrinsic variability of each algorithm itself, allowing us to focus solely on the impact of the chosen algorithms on the navigation task. While performance metrics suggest some convergence between DQN, REINFORCE, and the Hybrid method in certain environments, the latter consistently achieved results closer to the optimal solution. This indicates that the proposed Hybrid approach offers a promising avenue for improving navigation performance in mobile robots.

18 Conclusion

This paper presented a novel hybrid reinforcement learning algorithm, Combined DQN-REINFORCE, that integrates improvements from both value-based (DQN) and policy-based (REINFORCE) approaches. We demonstrated that this hybrid method achieves faster learning and superior reward compared to traditional RL algorithms in a grid-world navigation task.

Beyond the proposed method, we conducted a comprehensive evaluation including five additional value-based and one policy-based algorithm across various navigation scenarios. The consistent outperformance of Combined DQN-REINFORCE across cluttered environments, mazes, and moving obstacles highlights its generalizability and potential for real-world mobile robot navigation tasks.

Our work contributes significantly to the field of RL for autonomous mobile robots. The comparative study underscores the importance of algorithm selection for specific navigation challenges. These findings offer valuable insights for researchers and developers aiming to enhance the navigation capabilities of autonomous mobile robots.

References

[1] Aqeel Anwar and Arijit Raychowdhury. "Autonomous Navigation via Deep Reinforcement Learning for Resource Constraint Edge Nodes Using Transfer Learning". In: *IEEE Access* 8 (2020), pp. 26549–26560. DOI: 10.1109/access.2020.2971172.

[2] Gang Chen et al. "Coordinating Multiple Agents via Reinforcement Learning". In: *Autonomous Agents and Multi-Agent Systems* 10.3 (2005), pp. 273–328. DOI: 10.1007/s10458-004-4344-3.

[3] Artur Dubrawski and James L. Crowley. "Learning locomotion reflexes: A self-supervised neural system for a mobile robot". In: *Robotics and Autonomous Systems* 12.3-4 (1994), pp. 133–142. DOI: 10.1016/0921-8890(94)90020-5.

[4] Rafet Durgut, Mehmet Emin Aydin, and Abdur Rakib. "Transfer Learning for Operator Selection: A Reinforcement Learning Approach". In: *Algorithms* 15.1 (2022), p. 24. DOI: 10.3390/a15010024.

[5] Michael Ganger, Ethan Duryea, and Wei Hu. "Double Sarsa and Double Expected Sarsa with Shallow and Deep Learning". In: *Journal of Data Analysis and Information Processing* 04.04 (2016), pp. 159–176. DOI: 10.4236/jdaip.2016.44014.

[6] Hado V Hasselt. "Double Q-learning". In: *Advances in Neural Information Processing Systems (NIPS)*. 2010, pp. 2613–2621.

[7] F. S. He et al. "Learning to play in a day: Faster deep reinforcement learning by optimality tightening". In: *International Conference on Learning Representations (ICLR)*. 2017.

[8] Fumihiko Ishida et al. "Reinforcement-learning agents with different temperature parameters explain the variety of human action–selection behavior in a Markov decision process task". In: *Neurocomputing* 72.7-9 (2009), pp. 1979–1984. DOI: 10.1016/j.neucom.2008.04.009.

[9] Yesmina Jaafra et al. "Reinforcement learning for neural architecture search: A review". In: *Image and Vision Computing* 89 (2019), pp. 57–66. DOI: 10.1016/j.imavis.2019.06.005.

[10] Reza Jafari and Mohammad Masoud Javidi. "Solving the protein folding problem in hydrophobic-polar model using deep reinforcement learning". In: *SN Applied Sciences* 2.2 (2020). DOI: 10.1007/s42452-020-2012-0.

[11] Rkaa Kamil, Mohamed Mohamed, and Bashra Oleiwi. "Path Planning of Mobile Robot Using Improved Artificial Bee Colony Algorithm". In: *Engineering and Technology Journal* 38.9 (2020), pp. 1384–1395. DOI: 10.30684/etj.v38i9a.1100.

[12] Makoto Katoh. "Maze Search Using Reinforcement Learning by a Mobile Robot". In: *Advances in Robotics Mechanical Engineering* 1.2 (2018). DOI: 10.32474/arme.2018.01.000110.

[13] Ganesh Khekare and Shahrukh Sheikh. "Autonomous Navigation Using Deep Reinforcement Learning in ROS". In: *International Journal of Artificial Intelligence and Machine Learning* 11.2 (2021), pp. 63–70. DOI: 10.4018/ijaiml.20210701.oa4.

[14] Xiali Li et al. "A Reinforcement Learning Model Based on Temporal Difference Algorithm". In: *IEEE Access* 7 (2019), pp. 121922–121930. DOI: 10.1109/access.2019.2938240.

[15] Vinh Nguyen and Jeremy Marvel. "Evaluation of Robot Degradation on Human-Robot Collaborative Performance in Manufacturing". In: *Smart and Sustainable Manufacturing Systems* 6.1 (2022), p. 20210036. DOI: 10.1520/ssms20210036.

[16] Omid Nobari, Arash Nobari, and Reza Eslamipoor. "Optimal Point-to-Point Path Planning of Manipulator by Using Vibration Damping Optimization Algorithm and Game Theory Method". In: *Journal of testing and evaluation* 47.4 (July 2019), pp. 2867–2888. DOI: 10.1520/jte20180332. URL: https://doi.org/10.1520/jte20180332.

[17] Olumide Obe and Ioan Dumitrache. "Adaptive Neuro-Fuzzy Controler With Genetic Training For Mobile Robot Control". In: *International Journal of Computers Communications Control* 7.1 (2012), p. 135. DOI: 10.15837/ijccc.2012.1.1429.

[18] Anish Pandey. "Mobile Robot Navigation and Obstacle Avoidance Techniques: A Review". In: *International Robotics Automation Journal* 2.3 (2017). DOI: 10.15406/iratj.2017.02.00023.

[19] B.K. Patle et al. “A review: On path planning strategies for navigation of mobile robot”. In: *Defence Technology* 15.4 (2019), pp. 582–606. DOI: 10.1016/j.dt.2019.04.011.

[20] Xiaohong Peng et al. “Enhanced Autonomous Navigation of Robots by Deep Reinforcement Learning Algorithm with Multistep Method”. In: *Sensors and Materials* 33.2 (Feb. 2021), p. 825. DOI: 10.18494/sam.2021.3050. URL: https://doi.org/10.18494/sam.2021.3050.

[21] “Reinforcement Learning:” in: *Kybernetes* 27.9 (1998), pp. 1093–1096. DOI: 10.1108/k.1998.27.9.1093.3.

[22] Ahmad EL Sallab et al. “Deep Reinforcement Learning framework for Autonomous Driving”. In: *Electronic Imaging* 29.19 (2017), pp. 70–76. DOI: 10.2352/issn.2470-1173.2017.19.avm-023.

[23] “Simulation of autonomous mobile robot using fuzzy logic”. In: *Journal of Al-Qadisiyah for Computer Science and Mathematics* 10.2 (2018). DOI: 10.29304/jqcm.2018.10.2.373.

[24] Narendra SinghPal and Sanjeev Sharma. “Robot Path Planning using Swarm Intelligence: A Survey”. In: *International Journal of Computer Applications* 83.12 (2013), pp. 5–12. DOI: 10.5120/14498-2274.

[25] R.S. Sutton and A.G. Barto. “Reinforcement Learning: An Introduction”. In: *IEEE Transactions on Neural Networks* 9.5 (1998), pp. 1054–1054. DOI: 10.1109/tnn.1998.712192.

[26] Richard S. Sutton, Doina Precup, and Satinder Singh. “Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning”. In: *Artificial Intelligence* 112.1-2 (1999), pp. 181–211. DOI: 10.1016/s0004-3702(99)00052-1.

[27] T. Tsuji et al. “A control design of robotics using the genetic algorithm”. In: *Artificial Life and Robotics* 2.1 (1998), pp. 24–27. DOI: 10.1007/bf02471148.

[28] Yuanliang Zhang, Cheng Chen, and Qing Liu. “Mobile Robot Path Planning Using Ant Colony Algorithm”. In: *International Journal of Control and Automation* 9.9 (2016), pp. 19–28. DOI: 10.14257/ijca.2016.9.9.03.

[29] Kai Zhu and Tao Zhang. “Deep reinforcement learning based mobile robot navigation: A review”. In: *Tsinghua Science and Technology* 26.5 (2021), pp. 674–691. DOI: 10.26599/tst.2021.9010012.